Engineering Predictable AI — Student Cheat Sheet (v6.2)

Bootcamp: Full-Stack & AI Bootcamp 2026 · Week 16 **Audience:** Full-Stack · Data Analytics · GenAI **Version:** v6.2 · Jul 2026 · foundations expanded · 3 hands-on · 10 pinned tips **Pin this near your screen.**

The reframe (why you signed up for prompt engineering but heard "context" all class)

> *"Prompt engineering was always a little bit ridiculous. Context is king."* — Tim Berglund, Confluent, 2026

Anthropic, Google, LangChain, and Confluent all agree: **a prompt is one lever inside a context system**. The industry now calls this **context engineering**. Same content, upgraded framing.

One-line mental model: the model is the CPU. The context window is RAM. Everything you'll do this class is memory management.

The whole class on one line

> **Three roles. Six ingredients. Four strategies (write / select / compress / isolate). One JSON schema you can ship.**

(The optional Claude Code bonus at the back of this sheet adds: *read deeply · write a plan · annotate until it's right · let the agent execute without stopping*.)

Foundation 1 — A prompt has THREE roles

Role	What	Color	Where you see it
SYSTEM	Setup rules. Persona. Format. Hard constraints.	teal	Workbench / Claude Code system message; HIDDEN in chat boxes
USER	What you want THIS turn. The question. The input.	blue	The text box you type into
ASSISTANT	Model's reply — and every prior reply in this session.	amber	Everything above your input box

> **Vocab:** Anthropic `system` = OpenAI `developer` = Google `system_instruction`. Same thing.

Foundation 1.5 — What you CAN and CANNOT control

YOU CONTROL	YOU INFLUENCE	YOU DON'T CONTROL
System prompt	Format compliance	Exact word choice
User message	Refusal behavior	Internal reasoning steps
Model choice	Tone / register	Whether it used your context vs training

YOU CONTROL	YOU INFLUENCE	YOU DON'T CONTROL
Temperature	Which sources it cites	Latency guarantees
max_tokens	How much reasoning it shows	Cost precision
Tools + schemas		Training cutoff date
Structured output schema		
Few-shot examples		

Debugging rule: when a prompt fails, ask — was it a CONTROL knob I forgot, or an INFLUENCE lever I can't force? Answer changes what you do next.

Foundation 2 — The 6-ingredient prompt

Audit any prompt against these. If you can't fill in 5 of 6 → rewrite.

1. **ROLE** — "You are a senior security engineer."
2. **CONTEXT** — "My audience is busy CFOs."
3. **TASK** — One verb. Classify. Extract. Summarize. (No "and"-and-"and".)
4. **CONSTRAINTS** — "≤ 80 words. No marketing language. Never invent stats."
5. **OUTPUT FORMAT** — Show a schema. `{ "label": ..., "confidence": ... }`
6. **EXAMPLES** — 0, 1, or a few. The model copies the *shape* more than the words.

Foundation 3 — The right altitude for a prompt

There's a Goldilocks zone. Miss it either way and the model breaks.

Altitude	Example	Why it breaks
TOO VAGUE	*"Be helpful and use the appropriate tools."*	Model improvises. Coin toss every run.
RIGHT ALTITUDE	*"Classify each message as SALES / SUPPORT / SPAM. Return JSON. Ask if empty."*	Defines outcome + shape. Model handles the how.
TOO PRESCRIPTIVE	*"IF billing AND refund AND $>100 AND day=Mon → call escalate_tool()"*	Every unlisted case breaks. Debugging spaghetti in prose.

The rule: define outcomes and shape. Let the model reason about the how.

The two knobs (temperature + max_tokens)

- **`temperature = 0`** for code, extraction, classification, JSON.
- **`temperature = 0.7`** for human-readable writing or brainstorming.
- **`max_tokens` always set.** A prompt without it is a runaway bill.
- **`top_p = 1`.** Don't touch it. Adjusting it AND temperature is a debugging nightmare.

Force JSON — three providers, three syntaxes

Provider	Native mode
Anthropic Claude	`tool_use` with `input_schema` (or `structured_outputs`). **PREFILL** assistant turn with { — Claude only.
OpenAI	`response_format: { type: "json_schema", strict: true, ... }`
Google Gemini	`response_mime_type: "application/json"` + `response_schema`

Universal fallback (works on any model): paste schema in system + ONE filled example + end the user turn with: *"Return ONLY valid JSON. No prose, no fences."*

Vibe phrases → engineered ones

Dimension	VIBE (fails)	ENGINEERED (works)
TONE	"Be professional."	"Write for a CFO who hates jargon."
LENGTH	"Make it short."	"Exactly 3 bullets, ≤ 20 words each."
MISSING INFO	"Don't hallucinate."	"If not in source, reply: `NOT_IN_SOURCE`"

Pattern: Vague adjective → name a specific PERSON, NUMBER, or ESCAPE HATCH.

Grounding — remove the chance to guess

```
SYSTEM:
  Answer ONLY using <source>.
  Quote the sentence that supports each claim.
  If not in <source>, reply EXACTLY: NOT_IN_SOURCE

USER:
  <source>
  {paste the document here}
  </source>

  Question: {the user question}
```

How to actually paste this — the `SYSTEM:` / `USER:` labels are a documentation shape. In every real playground, those are **separate fields**:

Tool	Where the System half goes	Where the User half goes
Claude.ai / ChatGPT chat	not visible (vendor sets it)	the chat box
Claude Workbench	"System" box (top-left)	"User" box (middle)
AI Studio	"System instructions" (right-side panel)	the prompt box at the bottom
OpenAI Playground	"developer" message slot	"user" message slot

Never paste the literal words `SYSTEM:` or `USER:` into a playground field. They're labels for you, not for the model.

Without the escape hatch, the model has no legal way to say "I don't know" — so it invents.

Context engineering — 6 components compete for space

#	Component	You control?
1	System prompt	YES
2	User message	YES
3	Tools (names + schemas)	YES
4	Retrieved resources (RAG / MCP)	GROWS
5	Assistant history	GROWS
6	Tool calls & responses	GROWS

60-70% rule — Aim for that capacity. When you fill the window, the **middle** of the context gets ignored ("lost in the middle"). Components 4-5-6 are usually the culprit.

Four strategies to engineer the context (LangChain framework)

Every technique in the industry fits into one of these four buckets. Memorize the verbs.

Strategy	Problem it solves	Concrete moves
WRITE	Agents forget.	Scratchpads · `claude.md` rules files · memory extraction.
SELECT	Not everything is relevant now.	Agentic RAG · RAG over tool defs · episodic/semantic/procedural memory.
COMPRESS	Context grows every turn.	Chunk + rerank · running summaries · drop old tool outputs.
ISOLATE	Contexts contaminate each other.	Sub-agents · phase resets · separate planner/coder sessions.

When the agent misbehaves: name which strategy you skipped. Fix that one.

Four failure modes (Drew Bruyne, 2025)

Named things debug faster than vibes.

Mode	Symptom	Fix (which strategy?)
POISONING	Hallucination or bad tool result propagates forward.	**PRUNE** — validate tool outputs, drop failed attempts.
DISTRACTION	Model over-relies on recent history, stops using training.	**COMPRESS** — summarize aggressively, reset if you can.
CONFUSION	Too many tools; model picks a nonsense one.	**SELECT** — dynamic tool set, RAG over tool descriptions.
CLASH	New info contradicts what's already in context.	**AUTHORITY ORDER** — system > retrieved > history. Declare it explicitly.

The 3-phase loop (workflow that puts it all together)

The most reliable pattern for any long AI task. Works in Claude Code, Cursor, or plain chat.

Phase	What you do	Artifact	Which strategies it uses
1. Research	Deep read. Map the terrain. No assumptions.	`research.md`	**Write** + **Isolate**
CTX RESET	Open a NEW window. Bring only research.md.	—	**Compress**
2. Plan	Architecture. Step list. Edge cases. Human annotates inline.	`plan.md`	**Write** + **Select**
CTX RESET	Open a NEW window. Bring only plan.md.	—	**Compress**
3. Implement	Execute the plan step by step. Typecheck. Don't deviate.	code	**Isolate**

Why the resets matter: each phase produces a compacted artifact. The next phase starts with a fresh window containing ONLY that artifact. Zero contamination. Zero context rot.

Rule: never let the model write code until you have reviewed a plan.md.

---

10 tips to pin to your wall

1. If your prompt has 3+ IF/THENs, rewrite as a goal.
2. `temperature = 0` for anything code parses. `0.7` for humans.
3. Always set `max_tokens`. Never leave it default.
4. Vague adjectives are bugs — replace with a specific person, number, or escape hatch.
5. Every production prompt needs an escape hatch (`reply NOT_IN_SOURCE`).
6. Static content up top. Prefix caching is 10x cheaper.
7. Aim for 60–70% context utilization. Fuller = middle ignored.
8. Name the failure mode before you fix (Poison / Distract / Confuse / Clash).
9. Reset context between phases. Fresh window per artifact.
10. Prompt is the atom. Master it before you scale.

Meta-tip. Name the primitive (prompt / context / workflow) before you touch it. Debugging speed × 5.